

Seguridad en APIs: constrúyela tú

Proyecto: MediReserva — Sistema de reservas de citas médicas

Cómo funciona este taller

Este es un taller de dos momentos. Hoy construyes la fundación y la autenticación del sistema (igual que hicimos con DevAcademy en clase, pero solo con tus manos). El próximo lunes recibirás la segunda parte para completar la autorización.

No hay código en este documento. Tienes el encargo del cliente, las reglas del negocio, tu misión, los comandos de arranque y pistas para guiarte. El código lo escribes tú, aplicando lo que ya sabes de biblioteca_api y DevAcademy.

No estás solo: puedes apoyarte en tus dos proyectos anteriores como referencia. Eso no es copiar, es lo que hace un desarrollador real: reutilizar lo que ya construyó.

1. El encargo — Un correo llega a tu bandeja

Eres desarrollador backend. Esta mañana recibes este correo de un cliente nuevo. Léelo con atención: cada frase esconde una decisión técnica que tendrás que tomar.

Nuevo mensaje — Cliente: Clínica MediReserva

De: Administración de Clínica MediReserva

Para: Equipo de desarrollo

Asunto: Necesitamos el backend de nuestro sistema de citas

Hola,

Somos una clínica y queremos digitalizar la gestión de citas médicas. Hoy todo lo hacemos en papel y es un caos. Necesitamos que ustedes construyan el backend.

El sistema funciona así: tenemos médicos que atienden en distintas especialidades, y pacientes que solicitan citas con ellos. Un paciente se registra, mira los médicos disponibles y agenda una cita. Cada médico define los horarios en los que atiende.

Para nosotros es fundamental la privacidad: un paciente solo debe ver SUS propias citas, nunca las de otro paciente. Y un médico solo gestiona SU agenda y las citas que le corresponden, no las de otros médicos. Necesitamos además un administrador que supervise todo el sistema y gestione a los usuarios.

Confiamos en su trabajo. Quedamos atentos.

— Administración, Clínica MediReserva

2. Antes de programar — Entiende el negocio

Un buen desarrollador no abre el editor de inmediato. Primero traduce lo que el cliente pide a decisiones técnicas. Ese es tu primer reto.

Tu primer ejercicio (5 minutos, antes de tocar código)

Lee el correo otra vez y responde para ti mismo:

- ¿Cuántos tipos de usuario hay? ¿Qué palabra clave del correo te lo dice?
- ¿Qué frase del cliente se traduce en 'autorización a nivel de dato' (que cada quien vea solo lo suyo)?
- ¿Qué entidades necesitas? ¿Cuál entidad conecta a los pacientes con los médicos?
- El cliente nunca dijo 'JWT' ni 'contraseña hasheada'. ¿Por qué vas a usarlos igual?

Si respondiste esas preguntas, ya tienes el diseño en la cabeza. Ahora sí, a construir.

3. Tu misión de HOY (Parte 1 del taller)

Hoy construyes la fundación y la autenticación de MediReserva. Al terminar, un usuario debe poder registrarse (con su contraseña protegida) e iniciar sesión recibiendo un token. La protección de endpoints por rol es para la Parte 2.

Lo que debes lograr hoy

- Montar el proyecto con la estructura por capas que ya conoces.
- Configurar los secretos en un archivo .env y conectar PostgreSQL.
- Modelar las entidades del negocio (mira las pistas para saber cuáles).
- Crear los schemas de entrada y salida (recuerda la regla de oro de las contraseñas).
- Montar la capa de seguridad (hasheo de contraseñas y creación de tokens).
- Implementar el registro y el login con JWT.
- Probar todo en Postman y confirmar en Swagger.

4. Comandos de arranque — Para que no pierdas tiempo

Te damos el setup listo para que te concentres en lo que importa: la lógica y la seguridad. Ejecuta estos comandos en PowerShell, en la carpeta donde guardas tus proyectos.

Paso 1 — Crear el proyecto, el entorno e instalar dependencias

```
mkdir medireserva_api
cd medireserva_api
py -m venv venv
venv\Scripts\activate
pip install fastapi "uvicorn[standard]" sqlalchemy "psycopg[binary]" "pydantic[email]"
pydantic-settings python-dotenv "python-jose[cryptography]" "passlib[bcrypt]"
bcrypt==4.0.1 python-multipart
```

```
pip freeze > requirements.txt
```

⚠ Recuerda esto de la clase

Fijamos bcrypt==4.0.1 a propósito. Las versiones nuevas rompen con passlib y el registro falla con un error sobre '72 bytes'. Ya lo vivimos en clase. Si te aparece, la solución la tienes en tus notas.

Paso 2 — Crear la estructura de carpetas y archivos

Adapta este comando a las entidades que decidiste en tu análisis. Esta es la forma de la estructura; los nombres de los archivos de entidades los defines tú según tu modelo.

```
mkdir app, app\models, app\schemas, app\services, app\routers, app\auth
New-Item app\__init__.py, app\main.py, app\database.py, app\config.py -ItemType File
New-Item .env, .gitignore -ItemType File
# Crea dentro de models, schemas, services y routers los archivos
# que correspondan a TUS entidades (mira las pistas del punto 6).
# No olvides los __init__.py en cada carpeta.
```

5. ¿Cómo sabes que lo lograste? — Checklist de hoy

Ve marcando cada punto. Si todos quedan marcados, cumpliste la misión de hoy.

Montaje y configuración

- ☐ El servidor levanta con 'uvicorn app.main:app --reload' y muestra 'Application startup complete'.
- ☐ Las tablas de tus entidades aparecen creadas en PostgreSQL (révisalas en pgAdmin/DBeaver).
- ☐ La SECRET_KEY y la conexión a la base están en el .env, no escritas en el código.
- ☐ El archivo .env está listado en .gitignore.

Autenticación

- ☐ Un usuario puede registrarse enviando nombre, email y contraseña.
- ☐ En la base de datos, la contraseña se ve hasheada (algo como \$2b\$12\$...), nunca en texto plano.
- ☐ La respuesta del registro NO devuelve la contraseña ni su hash.
- ☐ Registrar dos veces el mismo email devuelve un error controlado.
- ☐ El login con credenciales correctas devuelve un token de acceso.
- ☐ El login con contraseña incorrecta devuelve 401 con un mensaje que no revela si falló el email o la clave.

Pruebas

- ☐ Probaste el registro y el login en Postman.
- ☐ Confirmaste el mismo comportamiento en Swagger (/docs).
- ☐ Inspeccionaste el token en jwt.io y viste el id del usuario en el campo 'sub'.

6. Pistas — Si te atascas (no son código, son dirección)

Usa estas pistas solo si las necesitas. Intenta primero por tu cuenta y apóyate en tus proyectos `biblioteca_api` y `devacademy_api` como referencia.

Pista 1 — Las entidades que probablemente necesitas

💡 Piensa en el negocio

El correo habla de usuarios (médicos, pacientes, admin), de médicos que atienden en especialidades, de horarios de atención, y de citas que conectan a un paciente con un médico.

Pregúntate: ¿qué entidad es la que 'conecta' a un paciente con un médico en un momento dado? Esa entidad es la equivalente al Préstamo de biblioteca o a la Inscripción de DevAcademy. Reconoce el patrón.

Sugerencia de entidades: Usuario, Medico (o el perfil/datos del médico y su especialidad), Horario (los bloques en que un médico atiende) y Cita (la reserva de un paciente con un médico). Puedes empezar con un núcleo más simple y crecer.

Pista 2 — El campo que decide los permisos

💡 Diseña pensando en la seguridad que viene

El cliente dijo: 'un paciente solo ve SUS citas' y 'un médico solo gestiona SU agenda'. Para que eso sea posible, ¿qué llave foránea necesita guardar cada Cita? ¿De quién debe saber que es?

Igual que el curso guardaba `instructor_id` en DevAcademy, aquí cada cita debe saber a qué paciente y a qué médico pertenece. Esos campos serán la base de la autorización de la Parte 2. Ponlos desde hoy.

Pista 3 — El rol

💡 Recuerda la decisión de seguridad crítica

Tu entidad Usuario necesita un campo 'rol'. ¿Cuál debe ser el valor por defecto al registrarse? Piensa: ¿quién se registra solo en una clínica, el paciente o el médico?

Y la pregunta de seguridad: si el registro dejara elegir el rol, ¿qué pasaría si alguien envía 'rol': 'admin'? Por eso el rol se fija en el servidor, no lo decide quien se registra. Médicos y admin se crean por vía controlada.

Pista 4 — La regla de oro de las contraseñas

💡 Entrada y salida no son lo mismo

El schema de entrada (crear usuario) recibe la contraseña en texto plano. El schema de salida (respuesta) NUNCA la incluye, ni siquiera hasheada. Son dos schemas distintos, a propósito.

Y antes de guardar: la contraseña se hashea. Nunca se guarda tal cual. Al iniciar sesión, se compara el hash, no el texto.

Pista 5 — El login es especial

💡 No es un endpoint cualquiera

El login usa el formulario estándar de OAuth2, no JSON. En Postman, el body va como x-www-form-urlencoded, con los campos 'username' (donde pones el email) y 'password'. Es el detalle que más confunde: tenlo presente.

Pista 6 — El orden de construcción sugerido

- Config y base de datos primero (.env, config.py, database.py). Sin base, nada corre.
- Luego los modelos (las tablas). Impórtalos en main.py antes de crear las tablas, o no se crean.
- Después los schemas (los contratos).
- La capa de seguridad (hashear y crear tokens).
- El service de usuarios (registrar con hash) y el router de auth (registro + login).
- Levanta, prueba, celebra.

7. Preguntas para pensar (piénsalas mientras construyes)

Estas preguntas no tienen que entregarse, pero pensarlas te hace mejor desarrollador. La Parte 2 partirá de ellas.

Para reflexionar

1. Cuando un paciente inicie sesión, su token guardará su id. En la Parte 2, ¿cómo usarías ese id para mostrarle solo SUS citas y no las de otros?
2. El cliente quiere que un médico gestione solo SU agenda. Ya tienes (o deberías tener) un campo que dice de qué médico es cada horario. ¿Qué comparación tendrías que hacer para permitir o negar una edición?
3. ¿Por qué el registro fija el rol como paciente y no deja que la persona elija ser médico? ¿Qué pasaría en una clínica real si cualquiera pudiera registrarse como médico?

8. Cierre de hoy y qué viene

Si completaste el checklist, MediReserva ya sabe quién es cada quien: los usuarios se registran de forma segura e inician sesión con JWT. Eso es exactamente donde dejamos DevAcademy en su primera sesión.

Lo que queda para la Parte 2 (próximo lunes): proteger los endpoints, implementar los tres roles, y lograr que cada quien haga SOLO lo suyo — el paciente con sus citas, el médico con su agenda, el admin con todo. Además, el sistema de refresh tokens y el manejo de errores estándar.

Hoy MediReserva aprendió a reconocer a las personas. El próximo lunes, aprenderá a poner límites.